THE CODE EDITOR

bm Code

Write the Lua and .b code of your bm game on the console, compile, disassemble, run and read its errors.

Contents

What bm Code is 4

What it does 4

Why it matters 4

Good to know 4

Opening and saving 5

Projects and games 5

Opening a cartridge 5

A new project 5

Saving 5

Leaving 5

The screen 6

Fonts 6

Two pages 6

Modules in a project 6

The menu 6

Editing keys 8

Moving 8

Find and rename in the project 8

Changing 8

Pages, tabs and font 9

Running and helping 9

With the gamepad 10

Completion and the assistant 11

Word completion 11

The assistant (F6) 11

Help on an API function (Ctrl+Q) 11

The check of API names on save 12

Breakpoints (F8) 13

When the game stops 13

Playing riff with Ctrl+Enter . . . 14

Running the game and reading errors . . . 15

Running . . . 15

When the game fails . . . 15

Guided tutorials . . . 16

Write and run your first line . . . 16

Fix an error with the error line . . . 16

Use a breakpoint . . . 16

Tips and limits . . . 18

Open points . . . 19

README

What this is. The manual of bm Code, the editor you use to write the Lua and `.b` code of a game or standalone script on the console itself.

How to use it. Read the first chapters once, then follow the guided tutorials in order. The key table is there to come back to.

What is missing. Every gap is marked with a highlighted TO FILL tag; nothing is invented.

What bm Code is

bm Code is the code editor of the bm suite. It opens a cartridge (`.bm` / `.bme` in Lua, `.b` or hybrid Lua+ `.b`) or a standalone `/code-b/*.b` or `/code-lua/*.lua` file, lets you change it, check or disassemble it, and run it from the same screen.

What it does

- Several cartridges open at once, each in its own tab, and two pages can be shown side by side.
- Three fonts: 6x12, 8x14 and 8x16. The 6x12 one fits 106 columns.
- Syntax colours, undo and redo, find and replace, go to line.
- A word completion: while you type, the rest of the likeliest word appears and Tab writes it.
- An assistant on F6 that answers questions and writes code for you.
- Breakpoints on F8 and the debugger, with F5 to run.
- Ctrl+Enter plays the code as music (riff).
- Ctrl+Q (on the pad Y+X) shows every signature of the API name under the cursor, each with its text; on save, names that are not in the API are reported and their line numbers turn red.

Why it matters

You can write, try and fix a game without a computer: the loop is edit, F5, look, come back.

Good to know

- Only the **code** of a cartridge changes. Its sprites, map, sounds and cover stay as they are.
- bm Code is a program of the Pi Zero W. The kernel of the PowKiddy RGB30 does not offer it; there you write code in the quick editor of the bm SDK (F2).
- Inside the bm SDK you can open the project in bm Code from the F1 page (*OPEN IN*, key 1).

Opening and saving

Projects and games

bm Code edits only **projects**, files that end in `.bme`. A game (`.bm`) is read only: you can open it and read it, but never change it.

When you save a game, bm Code does not touch it. It writes an editable copy in `/bme`, `name.bme`, and from then on the tab works on the copy. The tab name changes to the new file.

Opening a cartridge

Press **Ctrl+O**. The list shows `+ New project / file...` and then the `.bme` projects of `/bme`, the `.bm` and `.bme` files of `/carts` and of the root of the card, and standalone scripts in `/code-b` (`.b`) and `/code-lua` (`.lua`). Move with the up and down arrows, press **Enter** to open. A file that is already open just shows its tab.

A new project

Press **Ctrl+N**. A list shows the Lua templates (`Empty`, `Platform 2D`, `Top-down 2D`, `Shooter 2D`, `Versus 2D`, `3D scene`) plus `.b Game` (100% `.b` native), `Hybrid Lua+.b` (`main.lua` + `main.b`), `File .b` (`/code-b/`) and `File .lua` (`/code-lua/`). Move with the up and down arrows, press **Enter** to choose, **Esc** to leave. Each multi-file Lua template is split in modules (tabs on the second row); `Hybrid Lua+.b` adds a `main.b` module tab. Then a line asks the file name (8.3 in `/bme` for cartridges, `/code-b` for `.b`, or `/code-lua` for `.lua`). If the file already exists, bm Code says so and suggests Ctrl+O. Press **Ctrl+S** to write the file.

Ctrl+T opens a new empty tab with no file: the first save asks for a name.

Saving

Key	What it does
Ctrl+S	saves the tab; a tab with no file asks for a name
Ctrl+Shift+S	save as: asks for an 8.3 name in `/carts`
Ctrl+W	closes the tab; if it has changes it asks `Save`, `Discard` or `Cancel`

A tab with changes shows a `*` after its name. F5 saves every tab with changes that has a file before it runs the game; a new tab with no name is not saved.

Leaving

Choose **Exit** in the menu (Esc). With changes not saved it asks `Save all`, `Keep for later` or `Cancel`. **Ctrl+Esc** goes back to the bm menu; if there are changes, press Ctrl+Esc a second time and they are kept for next time.

The tabs, the cursors, the font and the breakpoints come back the next time you open bm Code. Text not saved is kept too, within a limit: see Tips and limits.

The screen

[TO FILL: shots/editor-code-screen.png]

- **Top bar**: the name `bm Code` is the first item at the left. Then one tab for each cartridge, with its file name. A `*` means changes not saved. On the right, `held: keys` reminds you that F12 held shows every key.
- **The code**: line numbers on the left, the current line lit. A breakpoint is a dark red line number. The line of an error has a reddish background.
- **Status line** (the last row): the file, a `*` if changed, then `ln` (line) of the total, `col` and the font. During music it adds `MUSIC` and the number of patterns playing. Messages appear next to the file name.

Fonts

F10 changes the font in turn: 6x12, 8x14, 8x16. The status line shows the one in use.

Two pages

F4 shows two pages side by side. Each page has its own tab. **F7** moves between the pages; **F2** and **F3** change the tab of the page you are in. In the top bar a tab shows `1` or `2` for the page that shows it. F4 again goes back to one page.

Modules in a project

A big program can be cut into **modules**. A module is a part of the code that starts at a line `--@module name`. The line is a Lua comment, so the game does not change: a project still has ONE code section, and the game built from it has the same bytes as before. The tabs are only for reading.

When a project has modules, a second row appears under the top bar. It shows the word `modules` and one tab for each module. The text before the first marker is the module `main`.

Key or click	What it does
Ctrl+P / Ctrl+Shift+P	next / previous module
click on a module tab	shows that module
right click on a module tab	the module menu
Esc, Modules...	the same menu

The module menu has **New module...** (asks a name, adds it after the one you are in), **Add / Open main.b** (opens or adds the native `.b` section of a hybrid project as a `main.b` module tab), **Rename module...** and **Merge into the one before** (the lines go to the end of the previous module and the marker line goes; no code is lost).

A name has letters, digits, `_`, `.` or `-`, up to 24 characters. Each module keeps its own cursor, undo and breakpoints. Ctrl+S, F5 and Ctrl+W work on the whole project, not on one module. A middle click on a module tab does nothing; a middle click on a file tab closes the file. When the game stops with an error, bm Code shows the module and the line in it.

The menu

Esc opens the menu (with a selection active, Esc first drops the selection). Up and down move, Enter chooses, Esc closes. The entries:

Entry	Key
New project	Ctrl+N
Open...	Ctrl+O
Save	Ctrl+S
Save as...	Ctrl+Shift+S
Close tab	Ctrl+W
Run the game	F5
Breakpoint	F8
Debug from the start	
Clear breakpoints	
Modules...	Ctrl+P
Split screen	F4
Font size	F10
Find	Ctrl+F
Find in the project...	Ctrl+Shift+F
Rename a word...	Ctrl+Shift+H
Replace	Ctrl+H
Go to line	Ctrl+L
Assistant	F6
Explain the error	F9
Compile / Check	
Disassemble .b	Ctrl+Shift+D
Play the music (riff)	Ctrl+Enter
Stop the music	Ctrl+.
Word completion	on / off
Words in comments	the language of the words
Pad typing	the state
Keys	F12 held
Exit	

When the SDK opened the file, the menu also has **Back to bm SDK**: it saves and returns.

Editing keys

These are the keys that exist in bm Code. **F11** and **F12** belong to the system and never reach the editor. A key list is on the screen while you hold **F12**. bm Code keeps its own F12 chip in the top bar.

Moving

Key	What it does
Arrows	move the cursor
Home	goes to the first character of the line; again, to column 1
End	goes to the end of the line
PgUp / PgDn	move by a page
Ctrl+L	go to line: type the number
Ctrl+F	find: type the text (the word under the cursor is suggested)
Ctrl+G	find the next
Ctrl+Shift+F	find a text in every tab and module of the project
Ctrl+Shift+H	rename the word under the cursor in every tab and module
Ctrl+H	replace all: asks the text, then the new text

Find and rename in the project

Ctrl+Shift+F (menu: *Find in the project...*) finds a text in every open tab and in every module of a project. Type the text. The word under the cursor is suggested. The search does not care about upper and lower case. A list opens with one line for each hit: `module:line text`. If you have more than one tab open, the name starts with the tab name. The list shows 200 hits at most; the title says when there are more. Move with the up and down arrows. Press **Enter** to go to the hit: bm Code opens the tab and the module and puts the cursor on the text. If the text is not in the project, the status line says `not found in the project`.

Ctrl+Shift+H (menu: *Rename a word...*) renames the word under the cursor. It changes the whole word only, in all modules and all open tabs. Text inside strings and comments stays as it is. Numbers stay too. If the cursor is not on a word, bm Code asks which word to rename. Then type the new name. It must be a valid Lua name: letters, digits and `_`, not a number first, not a keyword. The status line says how many words changed and in how many modules.

> **BEWARE**
> Rename does not check the scope. If two different variables have the same name, both change. Look at the result before you save.

Each module that changes gets one undo step. Press **Ctrl+Z** in each module to go back.

Changing

Key	What it does
Enter	new line, with the indent of the one above (two more after `then`, `do`, `else`, `function ...` and the like)
Backspace / Del	delete before / after the cursor
Tab	writes the grey-blue completion if there is one; if not, indents the line (or the selected lines) by two spaces
Ctrl+U	unindents the line (or the selected lines)
Ctrl+B	starts a selection; move the cursor to extend it; Ctrl+B again drops it
Ctrl+C / Ctrl+X	copy / cut the selection
Ctrl+V	paste; a whole line goes above the current one
Ctrl+K	cut the whole line
Ctrl+D	duplicate the line
Ctrl+Z / Ctrl+Y	undo / redo

Typing `end`, `else`, `elseif`, `until` or `}` alone on a line brings it back to the indent of its block.

Pages, tabs and font

Key	What it does
Ctrl+T	new empty tab
Ctrl+W	close the tab
F2 / F3	previous / next tab
Ctrl+P	next module of the project (Ctrl+Shift+P: before)
F4	one page / two pages
F7	move to the other page
F10	font 6x12 / 8x14 / 8x16

Running and helping

Key	What it does
F5 or Ctrl+R	run the game
F8	breakpoint on the line
F6	the assistant
F9	explain the error of the game
Ctrl+Shift+D	compile and disassemble `.b` into ARMv6 + VFPv2 instructions

Key	What it does
Ctrl+Q	help on the API function under the cursor
Ctrl+Enter	play the code as music (riff)
Ctrl+.	stop the music
Esc	drop the selection, or open the menu
Ctrl+Esc	back to the bm menu
F12 held	the keys of bm Code, under the system's

With the gamepad

Button	What it does
D-pad	moves the cursor
Y + left / right	previous / next tab
Y + up / down	page up / page down
X	the assistant
Y + X	help on the API function under the cursor
Start	the menu

In the menu and in the lists, the D-pad moves, A chooses and B goes back. The pad can also type text: switch **Pad typing** in the menu. Select switches it on and off; the menu entry calls the same button Share.

Completion and the assistant

Word completion

While you type a word, the rest of the likeliest word appears in grey-blue after the cursor. The status line shows `Tab:` and the word, and the other choices after `also:`. Press **Tab** and the word is written (green until your next key). Keep typing and it goes away.

The words depend on where the cursor is:

- in `.lua` code: Lua's words, bm's functions and the names that are in the tab;
- in `.b` code: `.b` keywords (`fn` , `function` , `struct` , `const` , `enum`), types (`i32` , `int` , `f32` , `bool` , `string`), and built-ins (`clamp` , `lerp` , `flr` , `peek` / `poke` , graphics/sound functions);
- in a comment (after `--`) and in a string: ordinary words, in Italian or in English;
- after `#entry:` (see below): Italian and the questions for the assistant.

In the menu, **Word completion** turns it on and off, and **Words in comments** changes the language of the ordinary words. Both are kept for the next time.

The assistant (F6)

F6 opens the assistant panel in code mode, with the word under the cursor as context. It answers while you type your question. **Enter** inserts the code it proposes at the cursor, **Esc** closes it. Nothing runs while it is closed. Inserting is one undo step with Ctrl+Z.

In code mode the assistant answers with four kinds of entries of its knowledge base: **API** functions, **how-to** examples with code, Lua **error** messages and **tips**. Its answers about sprites, 3D models and music are in the other tools.

An `#entry:` line

You can ask for something without opening the panel. Write a line like this and press **Enter** on it:

```
#entry: ... what you want ... #
```

The assistant does it in place and the status line says what it did, with `(Ctrl+Z undoes)` . If it knows nothing for that request, the status line says so and the text stays.

Explaining an error

F9 opens the assistant on the error of the last run. If there is no error, the status line says `no error: F5 runs the game, F9 explains its error` .

Help on an API function (Ctrl+Q)

Put the cursor on the name of a bm function and press **Ctrl+Q** (on the pad, hold **Y** and press **X**). A box opens. It shows each signature of the name, with its text under it. A name can have more than one signature. At the bottom, the box lists the API manual files that describe the name (`docs/api/NAME.md`), one line for each file. Any key closes the box.

- On a name that is not in the API, the status line says `no API help for 'NAME'` .
- With no name under the cursor, it says `no name under the cursor` .
- With the mouse, the right button opens a menu that has *API help*.

The check of API names on save

When you save a tab whose file ends in `.bm`, `.bme` or `.lua`, bm Code looks for names that look like calls to the bm API but are not in it, as `scripts/api_lint.py` does.

- The status line says `API: N unknown name(s), line L: NAME` for the first one, and the line number of each line with an unknown name turns **red**.
- A name that you define yourself, in this tab or in any other open tab, counts as known.
- The red numbers go away as soon as you change the text (and with undo or redo). Save again to check again.
- A breakpoint line keeps its dark red number.

TRY IT

Press **Ctrl+T** for an empty tab and type `cls(1)`. Put the cursor on `cls` and press **Ctrl+Q**: you see `cls([c])` and what it does. Press any key. Now type `sprt(1)` on a line and press **Ctrl+Q** on it: the status line says `no API help for 'sprt'`.

Breakpoints (F8)

A breakpoint makes the game stop on a line, so you can look inside it.

- **F8** on a line puts a breakpoint there: its line number turns dark red and the status line says `breakpoint on line N`. F8 again takes it away.
- **Clear breakpoints** in the menu removes them all from the tab.
- **Debug from the start** in the menu runs the game and stops it at its first line.
- Breakpoints follow their lines when you insert or delete lines above them. If you delete the line, its breakpoint goes.
- Breakpoints are kept with the session.
- In the code of a game, `breakpoint()` stops the debugger on that line when the game is run from bm Code. In other games it does nothing.

When the game stops

The debugger covers the screen. It shows the code around the line, the variables of the function (the locals, then the upvalues in light blue) and the stack.

Key	What it does
F10 or A	next line (over the calls)
F8 or X	into the call
Shift+F8 or Y	out of the function
F5 or Start	go on
Esc or Select	stop the game
Arrows	scroll the variables and choose the function of the stack

When you stop the game, bm Code comes back with the cursor on the line and the message `stopped in the debugger at line N`.

Playing riff with Ctrl+Enter

riff is bm's music notation (see the riff manual). In bm Code you write it and hear it at once.

- In a tab that has no `code [[ ]]` block, **Ctrl+Enter** plays the whole tab.
- In a game, it plays the `riff.code [[ ... ]]` block under the cursor, or the first one if the cursor is not in a block.
- The words of the notes that are playing **light up**, as long as the text is the one that was played.
- The status line says `MUSIC` and the number of patterns playing.
- **Ctrl+.** stops the music. The menu has the same two entries: *Play the music (riff)* and *Stop the music*.
- If the riff has an error, its line is marked and the status line says why. The music that was playing goes on.
- If nothing plays, the status line suggests giving a pattern to a name, for example `drums = s "kick*4"` .
- Running the game with F5 stops the music.

Running the game and reading errors

Running

F5 (or **Ctrl+R**, or *Run the game* in the menu) saves the tabs that have changes and runs the current tab:

- on a `.bm` or `.bme` cartridge (Lua, 100% `.b` or hybrid Lua+ `.b`), it compiles any `.b` code first and starts the cartridge (`Start + Select` or `Ctrl+Esc` returns);
- on a standalone `/code-b/*.b` file, it compiles natively and either runs the interactive `_init` / `_update` / `_draw` preview (`Esc` stops) or executes the function on the cursor line and prints its return value and timing;
- on a standalone `/code-lua/*.lua` file, it runs the script inside a sandbox (`tool_lua_run` , without `dofile` / `loadfile` / `load`).

When the game fails

If the game stops with an error, bm Code comes back and:

- puts the cursor on the line of the error (read from `main.lua:N:` or `main.b:N:` in the message, switching to the `main.b` module tab when needed);
- paints that line with a reddish background;
- shows `the game stopped:` and the message in the status line, followed by `(F9: explain)` .

Press **F9** to have the assistant explain it. The mark goes away the next time you run the game.

> **NOTE**
> The line number comes from the message. If the message has no line, the cursor stays where it was.

Guided tutorials

Start from bm Code. On the console it is the entry **bm Code** of the bm menu. You can also start from the bm SDK: F1, then 1 (*OPEN IN* bm Code).

Write and run your first line

1. Start bm Code. On the first start it shows a small starter program in an empty tab.
2. Press **Ctrl+N**. A line asks for the file name and suggests `game1.bme`. Press **Enter**. The status line says `new: Ctrl+S writes /bme/game1.bme`.
3. Press **Ctrl+S**. The status line says `saved /bme/game1.bme`.
4. Move the cursor to the line `print("hello!", 296, 40, 0xFFFFFF)` with the arrows. Change `hello!` into your own words. The `*` appears after the file name.
5. Press **F5**. The game starts: a yellow circle, your words on the screen, and the d-pad moves the circle.
6. Press **Start + Select** (or **Ctrl+Esc**) to leave. bm Code comes back with `back from the game`.

[TO FILL: shots/editor-code-first-run.png]

> TRY IT
> Add a line under the `print` : `print("score 0", 296, 60, 0xFFFFFF)`. Press F5 and look for it on the screen.

Fix an error with the error line

1. With the project of the first tutorial open, find the line `circfill(x, y, 12, 0xFFD050)` in `_draw`.
2. Break it on purpose: remove the last letter of `circfill`, so it reads `circfil(x, y, 12, 0xFFD050)`.
3. Press **F5**. The game stops as soon as it draws.
4. bm Code comes back. The cursor is on the broken line, which has a reddish background, and the status line says `the game stopped:` and the message, with `(F9: explain)`.
5. Press **F9**. The assistant explains the error. **Esc** closes it.
6. Put the letter back: `circfill`. Press **F5**. The circle is back and the mark is gone.

[TO FILL: shots/editor-code-error-line.png]

> BEWARE
> If you did not change the starter program, the broken line is line 16.

Use a breakpoint

1. With the project of the first tutorial open, move the cursor to the line `if btn(0) then x = x - 2 end` in `_update`.
2. Press **F8**. The line number turns dark red and the status line says `breakpoint on line N: F5 runs the game, it stops there`.

3. Press **F5**. The game starts and stops on that line. The debugger covers the screen: the code around the line, the variables, the stack.
4. Press **F10** to go to the next line. Look at the variables change.
5. Press **F5** to go on. `_update` runs every frame, so it stops again on the same line.
6. Press **Esc** to stop the game. bm Code comes back with the cursor on that line and says `stopped in the debugger at line N` .
7. Press **F8** on the line to take the breakpoint away.

[TO FILL: shots/editor-code-debugger.png]

> **TRY IT**
> Put a breakpoint inside `_draw` , press F5, and press F10 line by line. Which variable changes?

Tips and limits

- **Save with a short name.** A project file name is 8.3: `mygame.bme` .
- **Work on the copy.** If you open a game (`.bm`), the first save makes `name.bme` ; keep editing that one.
- **Tokens.** The tokens of a program are counted as the dev kit counts them; `code_tokens(text)` gives the number for a piece of Lua. They are information, not a limit: bm puts no ceiling on tokens. bm Code does not show them: the bm SDK does, on its F1 page and in the status line of its quick editor.
- **Text not saved is kept in a limited space.** When you leave or run the game, bm Code keeps the text of the tabs with changes (and of the tabs with no file) for the next time, up to 16000 characters in all; text beyond that is not kept. Save often.
- **Undo** remembers the last 100 steps of each tab (each module has its own).
- **Modules are one file.** Modules are lines `--@module name` in the one code section; they are not separate files on the card. A line with that exact form inside a long Lua string also starts a module for bm Code. The bm SDK shows the whole code without module tabs.
- **Tabs.** The top bar shows as many tabs as fit; the rest are shown as `...` . Use F2 and F3 to reach them.
- **Debug only from bm Code.** `breakpoint()` stops only in a game run from a tool such as bm Code.
- **Lines on the screen.** With one page on the 640x360 screen, the code shows 28 lines with the 6x12 font, 23 with the 8x14 and 20 with the 8x16 (the rows of the screen, less the top bar and the status line). The columns are 106, 80 and 80. These numbers come from the layout in the code, not from a count on the screen. The pad typing box takes some rows.
- **[TO FILL: the biggest file bm Code can open; the code sets no limit of its own and none was measured]**

COPY AND PASTE

```
F5          -- run the game
F8          -- breakpoint on the line
F9          -- explain the error
Ctrl+Enter  -- play the riff
```

Open points

- **The screen**: picture to take: shots/editor-code-screen.png
- **Guided tutorials**: picture to take: shots/editor-code-first-run.png
- **Guided tutorials**: picture to take: shots/editor-code-error-line.png
- **Guided tutorials**: picture to take: shots/editor-code-debugger.png
- **Tips and limits**: the biggest file bm Code can open; the code sets no limit of its own and none was measured